



Minimização de blocos consecutivos em larga escala por meio do algoritmo Lin-Kernighan-Helsgaun

Pedro Lucas Damasceno

Programa de Pós-Graduação em Ciência da Computação, Universidade Federal de Ouro Preto
Campus Morro do Cruzeiro, Ouro Preto, Minas Gerais, 35400-000, Brasil
`pedro.damasceno@aluno.ufop.edu.br`

Marco Antonio Moreira de Carvalho

Departamento de Computação, Universidade Federal de Ouro Preto
Campus Morro do Cruzeiro, Ouro Preto, Minas Gerais, 35400-000, Brasil
`mamc@ufop.edu.br`

RESUMO

O *Consecutive Blocks Minimization* (CBM) é um problema NP-difícil com aplicações relevantes em diferentes domínios. Este estudo investiga sua resolução por meio de uma redução ao problema do caixeiro viajante, combinada com o algoritmo *Lin-Kernighan-Helsgaun*, resultando no método LKH-CBM. Os experimentos foram conduzidos sobre o *benchmark* consolidado da literatura, composto por 85 instâncias, incluindo casos de grande porte. Os resultados mostram que o LKH-CBM obteve 71 novas melhores soluções, sem produzir resultados inferiores aos melhores valores previamente reportados. Além disso, o método encontrou soluções ótimas para todas as 42 instâncias com ótimo conhecido no subconjunto clássico e apresentou superioridade estatisticamente significativa em relação aos métodos estado da arte.

PALAVRAS CHAVE. Otimização Combinatória. Minimização de Blocos Consecutivos. Lin-Kernighan-Helsgaun.

Tópicos: OD, OA, MH

ABSTRACT

Consecutive Blocks Minimization (CBM) is an NP-hard problem with relevant applications in different domains. This study investigates a solution approach based on a reduction to the traveling salesman problem, combined with the Lin-Kernighan-Helsgaun algorithm, resulting in the LKH-CBM method. The experiments were conducted on a consolidated benchmark from the literature, composed of 85 instances, including large-scale cases. The results show that LKH-CBM obtained 71 new best solutions, with no results worse than the best values previously reported. Furthermore, the method found optimal solutions for all 42 instances with a known optimum in the classical subset and showed statistically significant superiority compared to state-of-the-art methods.

KEYWORDS. Combinatorial Optimization. Consecutive Blocks Minimization. Lin-Kernighan-Helsgaun.

Paper Topics: OD, OA, MH

1. Introdução

A análise de matrizes binárias é fundamental para a abstração e modelagem de problemas de otimização discreta e combinatória. Um problema canônico neste contexto, formalizado por Fulkerson e Gross [1965], é a verificação da propriedade dos blocos consecutivos de uns (*consecutive ones property* — C1P). Esta propriedade é satisfeita se existir uma permutação das colunas da matriz binária que resulte em apenas um bloco contíguo de elementos 1 em cada linha. A existência de tal permutação valida a consistência dos dados com um modelo estrutural perfeitamente linear, uma questão originalmente motivada pela análise da estrutura de genes. Como ilustração, considere a matriz M_1 (C1P), e a matriz M_2 , para a qual nenhuma permutação de colunas é capaz de satisfazer a C1P.

$$M_1 = \begin{bmatrix} 1 & 1 & 1 & 0 \\ 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 \end{bmatrix} \qquad M_2 = \begin{bmatrix} 0 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 \end{bmatrix}$$

Na prática, a maioria das matrizes que representam problemas reais não possui a C1P. Esta limitação motiva a formulação de um problema de otimização mais geral: o problema de minimização de blocos consecutivos (*consecutive blocks minimization* — CBM). O objetivo do CBM é determinar a permutação π de colunas da matriz binária que minimiza o número total de blocos, representando a melhor aproximação possível a uma estrutura linear.

A relevância do CBM é evidenciada por sua vasta aplicabilidade em domínios práticos distintos, incluindo solução de sistemas de equações, bioinformática [Alizadeh et al., 1995], compressão de dados [Lemire e Kaser, 2011] e organização de informações estruturadas em arqueologia [Kendall, 1969] e mapeamento genético, entre outras. Em tais contextos, a capacidade de lidar com instâncias de grande dimensão é fundamental, uma vez que dados reais frequentemente envolvem matrizes extensas e densas, impondo desafios computacionais significativos.

Trata-se de um problema NP-difícil [Kou, 1977], para o qual diferentes abordagens têm sido propostas na literatura, incluindo métodos exatos, heurísticas e meta-heurísticas. Entre estes, destacam-se métodos recentes considerados estado da arte, que combinam heurísticas tradicionais para solução do problema do caixeiro viajante (*traveling salesman problem*, TSP) e estratégias de intensificação baseadas em meta-heurísticas. Apesar desses avanços, a crescente disponibilidade de instâncias de maior dimensão evidencia limitações importantes dessas abordagens. Em particular, o desempenho de métodos se deteriora à medida que a dimensão das matrizes aumenta das dezenas para os milhares de colunas, exigindo camadas algorítmicas adicionais para obtenção de soluções de qualidade. Isso leva ao questionamento sobre até que ponto um algoritmo mais eficiente para o CBM pode, por si só, superar o estado da arte, sem a necessidade de estruturas algorítmicas adicionais.

Este estudo aborda essa questão ao investigar a aplicação do algoritmo *Lin-Kernighan-Helsgaun* (LKH) para a resolução do CBM, a partir de uma redução ao TSP. Diferentemente das abordagens existentes, que utilizam resolvidores do TSP como componente dentro de meta-heurísticas mais complexas, este estudo demonstra que o uso do LKH, mesmo com adaptações leves, é, neste contexto, mais determinante do que a incorporação de camadas meta-heurísticas adicionais.

Esses achados reforçam a importância da seleção adequada de algoritmos na resolução de problemas combinatórios e indicam que o método proposto constitui uma alternativa robusta e escalável para o CBM, especialmente em cenários que envolvem instâncias de grande dimensão.

Por fim, este estudo contribui ao demonstrar que avanços significativos podem ser obtidos por meio da reavaliação de ferramentas existentes, quando aplicadas de forma adequada ao CBM.

O restante deste estudo está organizado da seguinte forma. A Seção 2 formaliza o problema estudado. A Seção 3 apresenta uma revisão da literatura relevante. A Seção 4 descreve a abordagem proposta. A Seção 5 apresenta e discute os resultados computacionais. Por fim, a Seção 6 conclui o estudo e aponta direções para trabalhos futuros.

2. Definição do Problema

Considere uma matriz binária $A = [a_{ij}] \in \{0, 1\}^{m \times n}$, em que m e n denotam, respectivamente, o número de linhas e colunas. O CBM consiste em determinar uma permutação das colunas de A que minimize o número total de blocos consecutivos de 1 ao longo das linhas da matriz resultante. Seja Π o conjunto de todas as permutações de $\{1, \dots, n\}$. Para uma permutação $\pi \in \Pi$, denota-se por A^π a matriz obtida pela reordenação das colunas de A segundo π , isto é,

$$A^\pi = [a_{i,\pi(j)}]_{m \times n}.$$

Em cada linha de A^π , um bloco de uns corresponde a uma subsequência maximal de entradas iguais a 1 em posições consecutivas. Seguindo a formulação utilizada em Soares et al. [2020], o número total de blocos pode ser computado a partir do número de transições de 0 para 1. Para isso, considera-se uma coluna fictícia à esquerda da matriz, composta apenas por zeros. Assim, para todo $i \in \{1, \dots, m\}$, define-se $a_{i,\pi(0)} = 0$. O número de blocos na linha i da matriz permutada é dado por

$$b_i(\pi) = \sum_{j=1}^n a_{i,\pi(j)} (1 - a_{i,\pi(j-1)}),$$

e, conseqüentemente, o número total de blocos de 1 em A^π é

$$Z_{CBM}(\pi) = \sum_{i=1}^m \sum_{j=1}^n a_{i,\pi(j)} (1 - a_{i,\pi(j-1)}). \quad (1)$$

O objetivo do CBM é, portanto, encontrar uma permutação $\pi^* \in \Pi$ tal que

$$\pi^* \in \arg \min_{\pi \in \Pi} Z_{CBM}(\pi).$$

Para fins de ilustração, considere uma matriz binária $A_{5 \times 5}$, cuja disposição das colunas é dada por $\pi = [1, 2, 3, 4, 5]$. Observa-se que, na primeira linha, há dois blocos de uns, localizados nos intervalos de colunas $[1, 2]$ e $[5, 5]$. Na segunda linha, identificam-se dois blocos distintos, correspondentes às posições $[1, 1]$ e $[4, 5]$. A terceira linha contém um único bloco nas colunas $[3, 4]$. A quarta linha apresenta apenas um bloco na coluna 3, enquanto a quinta linha contém dois blocos, nas colunas 1 e 5, respectivamente. Dessa forma, a matriz A^π , apresenta um total de oito blocos de uns. É possível rearranjar as colunas da matriz A de acordo com a permutação ótima $\pi^* = [2, 5, 1, 4, 3]$. Na matriz em questão, os blocos se iniciam nas colunas 2, 5, 4, 3 e 5 em cada linha, respectivamente. O número total de blocos de uns em A^{π^*} é igual a cinco, três blocos a menos do que na matriz A^π .

Como o CBM é NP-difícil [Kou, 1977], abordagens exatas tornam-se impraticáveis em instâncias de maior porte, o que justifica o uso de estratégias heurísticas e meta-heurísticas. A próxima seção traz um breve apanhado do histórico de estudos sobre o CBM.

$$A^\pi = \begin{matrix} & 0 & 1 & 2 & 3 & 4 & 5 \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & \mathbf{1} & \mathbf{1} & 0 & 0 & \mathbf{1} \\ 0 & \mathbf{1} & 0 & 0 & \mathbf{1} & \mathbf{1} \\ 0 & 0 & 0 & \mathbf{1} & \mathbf{1} & 0 \\ 0 & 0 & 0 & \mathbf{1} & 0 & 0 \\ 0 & \mathbf{1} & 0 & 0 & 0 & \mathbf{1} \end{bmatrix} \end{matrix} \quad A^{\pi^*} = \begin{matrix} & 0 & 2 & 5 & 1 & 4 & 3 \\ \begin{matrix} 0 \\ 1 \\ 2 \\ 3 \\ 4 \end{matrix} & \begin{bmatrix} 0 & \mathbf{1} & \mathbf{1} & \mathbf{1} & 0 & 0 \\ 0 & 0 & \mathbf{1} & \mathbf{1} & \mathbf{1} & 0 \\ 0 & 0 & 0 & 0 & \mathbf{1} & \mathbf{1} \\ 0 & 0 & 0 & 0 & 0 & \mathbf{1} \\ 0 & 0 & \mathbf{1} & \mathbf{1} & 0 & 0 \end{bmatrix} \end{matrix}$$

3. Revisão bibliográfica

A origem do CBM está diretamente relacionada ao C1P. O estudo seminal de Fulkerson e Gross [1965] estabeleceu a base formal dessa propriedade e sua relação com grafos de intervalo. Posteriormente, Booth e Lueker [1976] propuseram um algoritmo com complexidade linear para o reconhecimento da C1P, e Dom [2009] apresentaram uma revisão abrangente sobre problemas associados a estruturas de blocos consecutivos em matrizes binárias. Nesse contexto, o CBM surge como uma generalização natural da C1P: em vez de decidir se uma ordenação perfeita existe, busca-se a permutação que minimiza a fragmentação dos blocos de uns.

Do ponto de vista de complexidade, o CBM é NP-difícil [Kou, 1977], permanecendo assim mesmo em casos restritos, como em matrizes $(*, 2)$, conforme Haddadi [2002]. Apesar de sua relevância prática, a literatura dedicada especificamente ao problema ainda é relativamente limitada. De modo geral, os métodos propostos podem ser organizados em duas linhas principais: abordagens que exploram sua estreita relação com o TSP e abordagens que tratam o CBM diretamente em sua formulação original.

A primeira linha de pesquisa é predominante na literatura, na qual cada coluna da matriz é associada a um vértice, e o custo de transição entre duas colunas é dado pela distância de Hamming [Hamming, 1950]. Essa conexão aparece de forma explícita em Alizadeh et al. [1995], que modelam o problema de mapeamento físico de cromossomos por meio de uma formulação equivalente ao CBM e demonstra a correção da redução ao TSP. Em Johnson et al. [2004], a mesma ideia é empregada no contexto de compressão e acesso eficiente a matrizes booleanas. Já Haddadi e Layouni [2008] propõem uma redução em duas etapas, passando pelo problema de minimização de blocos circulares, e mostram que, sob desigualdade triangular, uma heurística 1.5-aproximativa para o TSP, como a de Christofides [1976], fornece uma aproximação de mesma razão para o CBM.

Em paralelo, também foram desenvolvidas abordagens que atuam diretamente sobre a permutação de colunas da definição original do CBM. Haddadi et al. [2015] introduziram um algoritmo de busca local baseado em duas estruturas de vizinhança complementares, com movimentos avaliados em tempo $O(m)$ e vizinhanças de tamanho $O(n^2)$. Além da simplicidade e robustez, esse estudo estabeleceu uma base importante para o uso de estratégias de intensificação e diversificação inspiradas em *iterated local search* (ILS). Em seguida, Soares et al. [2020] propuseram uma representação em grafos centrada nas relações entre linhas da matriz binária, combinada com uma heurística construtiva gulosa e uma ILS, além de obter pela primeira vez as soluções ótimas via redução ao TSP com o uso do resolvidor exato Concorde, desenvolvido para o TSP originalmente.

Os avanços mais relevantes para o estado da arte, contudo, decorrem de métodos que combinam meta-heurísticas com o algoritmo *Lin-Kernighan* (LK) [Lin e Kernighan, 1973]. Em Haddadi [2021], o LK é utilizado na geração de soluções iniciais de alta qualidade para uma ILS enriquecida com novos operadores e um procedimento de avaliação parcial em tempo $O(1)$. Em Haddadi [2022], a busca local é reformulada por meio de uma *exponential neighborhood search* (ENS), na qual pequenos TSPs são resolvidos ao longo da exploração da vizinhança, tornando o LK um componente ainda mais central do método. Esses dois estudos representam o estado da

arte recente para o CBM, tanto em qualidade de solução quanto em desempenho computacional ao abordar instâncias de maior dimensão, com milhares de colunas, que limitaram a aplicação dos métodos anteriores, sobretudo o exato.

Apesar desse avanço, permanece uma lacuna importante. A literatura recente mostra que o uso do LK é decisivo para a qualidade das soluções, mas o resolvidor adotado continua sendo uma heurística clássica para o TSP, incorporada a camadas meta-heurísticas adicionais. Assim, ainda não está claro em que medida a substituição do LK por um algoritmo mais potente, como o LKH, pode por si só alterar o estado da arte do CBM, especialmente em instâncias de maior dimensão, nas quais a escalabilidade se torna um requisito central em aplicações práticas. Este estudo se insere precisamente nessa lacuna ao investigar a aplicação direta do LKH ao CBM por meio da redução ao TSP.

4. Metodologia

A abordagem adotada neste estudo baseia-se na redução do CBM a uma instância do TSP simétrico, seguida da resolução direta da instância reduzida pelo algoritmo LKH. A escolha dessa estratégia decorre de dois fatores: a forte correlação estrutural entre os dois problemas e a elevada qualidade das soluções produzidas pelo LKH para o TSP [Helsgaun, 2000].

4.1. Redução do CBM ao TSP

Dada uma matriz binária $A \in \{0, 1\}^{m \times n}$, constrói-se uma instância simétrica do TSP na qual cada coluna de A corresponde a um vértice. O custo associado a cada aresta é definido pela distância de Hamming [Hamming, 1950] entre as colunas correspondentes, isto é, pelo número de linhas em que seus valores diferem. Intuitivamente, colunas com padrões semelhantes tendem a ser posicionadas de forma adjacente na solução do TSP, o que favorece a formação de blocos mais longos de 1 e, conseqüentemente, reduz o número total de blocos na matriz permutada.

Como exemplo, considere novamente a matriz A^π . Ao compararmos as colunas 1 e 2, observam-se duas inversões, nas linhas 2 e 5, o que resulta em distância 2. Já entre as colunas 1 e 3 há inversões em todas as linhas, de modo que a distância correspondente é 5. O mesmo procedimento é aplicado a todos os pares de colunas para construir a matriz de distâncias C . O resultado deste processo é ilustrado à seguir. À esquerda, apresenta-se a matriz binária A^π ; à direita, a matriz C obtida a partir das distâncias entre suas colunas.

$$A^\pi = \begin{bmatrix} 0 & \mathbf{1} & \mathbf{1} & 0 & 0 & \mathbf{1} \\ 0 & \mathbf{1} & 0 & 0 & \mathbf{1} & \mathbf{1} \\ 0 & 0 & 0 & \mathbf{1} & \mathbf{1} & 0 \\ 0 & 0 & 0 & \mathbf{1} & 0 & 0 \\ 0 & \mathbf{1} & 0 & 0 & 0 & \mathbf{1} \end{bmatrix} \quad \Rightarrow \quad C = \begin{bmatrix} 0 & 3 & 1 & 2 & 2 & 3 \\ 3 & 0 & 2 & 5 & 3 & 0 \\ 1 & 2 & 0 & 3 & 3 & 2 \\ 2 & 5 & 3 & 0 & 2 & 5 \\ 2 & 3 & 3 & 2 & 0 & 3 \\ 3 & 0 & 2 & 5 & 3 & 0 \end{bmatrix}$$

Uma vez construída a matriz C , a instância de TSP pode ser resolvida por um algoritmo apropriado, produzindo um ciclo hamiltoniano. Como o TSP é um problema circular e o CBM exige uma ordenação linear, adiciona-se um vértice artificial associado a uma coluna nula. Esse vértice atua como marcador de início da permutação. Após a resolução do TSP, o ciclo hamiltoniano obtido é percorrido a partir desse vértice, que é então removido, produzindo uma ordenação linear das colunas originais.

A Figura 1 mostra o ciclo associado à permutação $\pi = [0, 1, 2, 3, 4, 5]$, com custo 16. Após a remoção do vértice artificial, essa ordenação produz uma solução com oito blocos de uns. Já a Figura 2 apresenta o ciclo correspondente à permutação ótima $\pi^* = [0, 2, 5, 1, 4, 3]$. Nesse caso, o custo do ciclo é 13 e a permutação resultante gera apenas cinco blocos de uns.

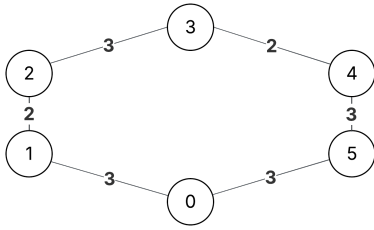


Figura 1: Ciclo hamiltoniano correspondente à permutação π .

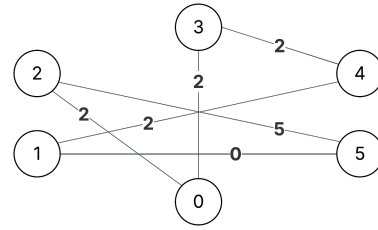


Figura 2: Ciclo hamiltoniano correspondente à permutação π^* .

4.2. Algoritmo de Lin-Kernighan-Helsgaun

O algoritmo LKH, proposto por Helsgaun [2000] é uma heurística de busca local avançada para o TSP simétrico, construída a partir da clássica heurística de k -opt variável de Lin e Kernighan [1973]. Resumidamente, métodos baseados em k -opt realizam movimentos de remoção de k arestas de uma solução e inserção de outras k arestas para reconstruir o ciclo hamiltoniano que representa uma solução. O LKH inclui mecanismos adicionais de seleção de arestas candidatas, avaliação de movimentos e reinicialização controlada. Em termos práticos, o método alterna entre uma fase de pré-processamento estrutural da instância e uma fase iterativa de melhoria da solução, repetida ao longo de múltiplas tentativas independentes.

A etapa de pré-processamento tem como objetivo restringir o espaço de busca às arestas mais promissoras. Para isso, o LKH executa inicialmente um procedimento de *ascent*, no qual penalidades associadas aos vértices são ajustadas por uma estratégia de subgradientes com base em 1-árvores. Esse processo produz duas informações importantes: um limitante inferior para o valor da solução do TSP e um conjunto de penalidades capaz de modificar os custos das arestas de modo a evidenciar estruturas mais favoráveis à busca local. A partir dessas penalidades, o algoritmo calcula a chamada α -proximidade, usada para construir conjuntos de arestas candidatas contendo, para cada vértice, apenas um subconjunto de arestas consideradas mais prováveis de pertencerem à solução ótima.

Após o pré-processamento, o LKH executa uma sequência de tentativas de melhoria da solução. Em cada tentativa, uma solução inicial é construída e submetida a um processo iterativo de melhoria baseado em trocas sequenciais do tipo k -opt. Diferentemente de um método com valor fixo de k , o LKH expande dinamicamente a sequência de trocas enquanto houver melhoria da solução. A busca parte de uma aresta da solução atual, remove arestas existentes e insere novas arestas preferencialmente escolhidas entre as candidatas previamente calculadas. A cada passo, o algoritmo avalia o ganho acumulado e verifica se a troca parcial pode ser completada de forma viável, respeitando a estrutura de um ciclo hamiltoniano.

Um aspecto importante da implementação é o uso do chamado *critério de melhor melhoria*. Em vez de aceitar apenas movimentos completos previamente definidos, o método permite prolongar uma sequência de trocas enquanto o ganho parcial continuar favorável e houver perspectiva de fechamento factível do movimento. Isso dá ao algoritmo um comportamento de profundidade variável, em que a vizinhança efetivamente explorada depende da qualidade das oportunidades encontradas durante a própria busca. Além disso, o LKH incorpora procedimentos de *patching* para tratar casos em que as trocas intermediárias geram subciclos, tentando recombina-los em uma única solução melhorada.

Quando uma melhoria é encontrada, a solução é atualizada e a busca reinicia a partir dela. Caso contrário, a tentativa é encerrada e uma nova solução inicial pode ser gerada, com pequenas

perturbações ou escolhas alternativas. Ao final de todas as tentativas, o algoritmo devolve a melhor solução encontrada. O mecanismo de múltiplas tentativas é importante porque reduz a dependência de uma única solução inicial e aumenta a robustez do método em instâncias maiores. O fluxo geral da implementação adotada para o LKH é resumido no Algoritmo 1.

Algoritmo 1 LKH

Entrada: Instância simétrica do TSP

Saída: Melhor solução T^* encontrada

```

1: Execute o procedimento de ascent para estimar penalidades dos vértices e obter uma 1-tree de referência
2: Calcule, para cada vértice, as medidas de  $\alpha$ -proximidade
3: Construa os conjuntos de candidatas a partir das arestas mais promissoras
4:  $T^* \leftarrow \emptyset$ 
5: para cada tentativa faça
6:   Gere uma solução inicial  $T$ 
7:    $melhorou \leftarrow$  verdadeiro
8:   enquanto  $melhorou$  faça
9:      $melhorou \leftarrow$  falso
10:    para cada aresta da solução  $T$  faça
11:      Inicie uma busca sequencial de trocas k-opt
12:      Expanda a sequência de trocas enquanto houver ganho acumulado promissor
13:      Restrinja inserções preferencialmente às arestas do conjunto de candidatas
14:      Verifique a viabilidade do fechamento da troca em um ciclo hamiltoniano
15:      Aplique patching, quando necessário, para recombinar subciclos
16:      se uma solução melhor  $T'$  for encontrada então
17:         $T \leftarrow T'$ 
18:         $melhorou \leftarrow$  verdadeiro
19:        Reinicie a busca sobre a nova solução
20:      fim se
21:    fim para
22:  fim enquanto
23:  se  $T^* = \emptyset$  ou  $custo(T) < custo(T^*)$  então
24:     $T^* \leftarrow T$ 
25:  fim se
26: fim para
27: retorne  $T^*$ 

```

4.3. LKH-CBM

O método proposto neste estudo, denominado LKH-CBM, combina a redução do CBM ao TSP com a aplicação direta do algoritmo LKH. Essa escolha é tecnicamente natural no contexto do CBM, pois os métodos estado da arte recentes já evidenciam a importância de heurísticas baseadas em *k-opt* para a obtenção de soluções de alta qualidade. A diferença central é a exploração direta de um algoritmo mais potente para o TSP, com o objetivo de verificar se essa substituição, por si só, é suficiente para deslocar o estado da arte do problema.

O fluxo de execução do LKH-CBM é resumido no Algoritmo 2. Inicialmente, adiciona-se à matriz binária uma coluna nula, correspondente ao vértice artificial. Em seguida, constrói-se a matriz de distâncias entre pares de colunas, utilizando-se a distância de Hamming. A instância simétrica de TSP resultante é então resolvida pelo algoritmo LKH. Por fim, o vértice artificial é identificado e removido da solução obtida, a sequência remanescente define a permutação linear das colunas da matriz original e o número total de blocos é calculado de acordo com a Equação 1.

Sob a perspectiva de implementação, o LKH-CBM foi concebido para manter simplicidade estrutural sem abrir mão de rigor computacional. A etapa específica do CBM concentra-se

Algoritmo 2 LKH-CBM

Require: Matriz binária $A \in \{0, 1\}^{m \times n}$

Ensure: Permutação π e valor $Z_{CBM}(\pi)$

- 1: Adicione a A uma coluna nula, correspondente ao vértice artificial;
 - 2: Construa a matriz de distâncias C entre pares de colunas usando distância de Hamming;
 - 3: Gere a instância simétrica de TSP associada a C ;
 - 4: Execute o LKH sobre a instância gerada;
 - 5: Obtenha o ciclo hamiltoniano retornado pelo LKH;
 - 6: Localize e remova o vértice artificial da solução;
 - 7: Defina π como a ordem linear dos vértices remanescentes;
 - 8: Aplique π às colunas de A ;
 - 9: Calcule $Z_{CBM}(\pi)$ pela contagem total de blocos consecutivos de uns;
 - 10: **retorne** π e $Z_{CBM}(\pi)$.
-

na geração da matriz de distâncias, na construção da instância reduzida e na recuperação da solução linear a partir do ciclo do TSP. Devido às dimensões das matrizes de entrada, optou-se por representá-las utilizando-se *bitsets* em C++, uma estrutura adequada e leve para minimizar o impacto em tempo computacional de sua manipulação. O cálculo da distância de Hamming é realizado pela aplicação do operador lógico XOR, o que pode ser realizado em tempo $O(n/w)$, em que n denota o número de colunas e w denota o tamanho da palavra do computador utilizado.

O LKH-CBM proposto adota uma configuração fixa ao longo de todos os experimentos, evitando calibração específica por família de instâncias e reforçando a comparabilidade dos resultados. Assim, todos os parâmetros do algoritmo foram mantidos constantes em cada execução, incluindo aqueles relacionados à intensidade dos movimentos locais, às estratégias de recombinação de ciclos e aos critérios de seleção de arestas candidatas, de modo a preservar um protocolo uniforme de avaliação. Os parâmetros definidos para a execução do LKH incluem (i) movimentos de até 5-opt na busca local; (ii) máximo de 3 subciclos a serem combinados durante o *patching*; (iii) máximo de dois vértices analisados em cada *patching*; e (iv) máximo de 5 arestas em cada conjunto de candidatas.

Essa característica também fortalece a interpretação dos resultados experimentais. Como o método não depende de mecanismos adicionais de refinamento nem de calibração altamente especializada, o desempenho do LKH-CBM pode ser atribuído de forma mais direta à qualidade da combinação entre a redução ao TSP e o algoritmo utilizado. Isso torna a contribuição do estudo mais nítida: verificar se, para o *benchmark* consolidado do CBM, a adoção de um algoritmo mais forte pode alterar de forma decisiva o patamar de qualidade das soluções, especialmente nas instâncias de maior porte, nas quais os métodos anteriores apresentam maior limitação prática.

5. Experimentos computacionais

Esta seção avalia o desempenho do LKH-CBM sobre o *benchmark* consolidado do CBM, formado por todas as instâncias empregadas na literatura desde 2015. Ao longo dos últimos anos, esse *benchmark* foi progressivamente ampliado com a inclusão de novas famílias de instâncias, o que elevou substancialmente sua dimensão e tornou a escalabilidade um critério cada vez mais relevante na comparação entre métodos. Nesse contexto, o objetivo desta avaliação é verificar, de forma direta e sob condições experimentais uniformes, em que medida o LKH-CBM é capaz de superar os métodos que definem o estado da arte recente para o CBM.

As instâncias estão organizadas em dois subconjuntos. O primeiro, denotado por $\{A, \dots, I\}$, reúne 45 instâncias com número de colunas variando entre 200 e 1000. O segundo, denotado por *SCP*, reúne 40 instâncias de maior porte, com 3000 a 10000 colunas. Todas as instâncias estão disponíveis na *Operations Research Library*.

Todos os experimentos foram conduzidos em um mesmo ambiente computacional, de modo a garantir comparabilidade estrita entre os resultados. As execuções foram realizadas em um processador Intel® Core™ i7-10700 a 2,90 GHz, com 16 GB de memória RAM DDR4 3200 MHz, sob Ubuntu 22.04.1. As implementações, exceto pelo ENS cujo código original foi disponibilizado pelo autor, foram compiladas com GCC 11.3.0, empregando as mesmas diretivas de otimização, em particular `-O3` e `-march=native`. Cada algoritmo foi executado 10 vezes por instância e, para todos eles, adotou-se um limite máximo de 7200 segundos por execução, sem mecanismos de interrupção forçada. Os códigos-fonte utilizados neste estudo, bem como todos os dados gerados nos experimentos, estão disponíveis em <https://github.com/pedroldm/cbm>.

5.1. Comparação com o estado da arte

A Tabela 1 reporta o resumo do desempenho dos métodos comparados, agregados para cada conjunto de instâncias. Para cada método comparado (ENS, ILS e LKH-CBM) são apresentados a melhor solução obtida (S^*) e o tempo médio de execução em segundos ($t(s)$). A distância percentual entre valores de solução (ou *gap*) é calculado por $gap_{REF}(\%) = \frac{LKH-CBM-REF}{REF} \times 100$, em que *REF* representa a melhor solução reportada por ENS ou ILS, conforme indicado. Os valores em negrito indicam os melhores resultados em cada grupo de instâncias.

Tabela 1: Resultados agregados por grupo de instâncias.

Grupo	ENS		ILS		LKH-CBM			
	S^*	$t(s)$	S^*	$t(s)$	S^*	$t(s)$	$gap_{ENS}(\%)$	$gap_{ILS}(\%)$
A	217,80	9,09	217,80	0,77	217,80	2,08	0,00	0,00
B	628,20	19,47	629,60	0,97	626,20	1,50	-0,32	-0,54
C	1262,40	17,21	1264,00	1,00	1259,60	1,06	-0,22	-0,35
D	472,80	62,32	472,80	4,87	472,80	23,60	0,00	0,00
E	1413,80	83,72	1416,00	4,58	1403,80	8,62	-0,71	-0,86
F	3006,40	78,67	3011,60	4,59	2997,20	8,99	-0,31	-0,48
G	897,00	273,21	898,20	17,48	896,80	159,21	-0,02	-0,16
H	2678,60	242,38	2687,60	11,79	2657,80	77,22	-0,78	-1,11
I	5730,40	245,70	5739,00	13,96	5696,80	33,85	-0,59	-0,74
SCPA	11631,40	3148,48	11595,60	199,04	11449,40	4711,77	-1,56	-1,26
SCPB	31869,60	4451,33	31762,20	267,00	31449,00	1069,26	-1,32	-0,99
SCPC	22245,40	10098,07	22170,20	558,81	21867,40	6142,98	-1,70	-1,37
SCPD	59542,60	8117,64	59299,40	573,83	58788,40	2048,18	-1,27	-0,86
SCPE	2288,80	28,30	2288,20	1,85	2284,60	4,48	-0,18	-0,16
SCPF	335312,80	5503,16	335193,00	551,36	334508,00	326,46	-0,24	-0,20
SCPG	161897,80	7300,66	161407,00	7327,55	159093,00	7253,43	-1,73	-1,43
SCPH	410088,80	7309,99	408465,20	6504,56	405908,40	7755,64	-1,02	-0,63

Os dados mostram que o LKH-CBM supera de forma ampla os melhores resultados previamente reportados para o *benchmark*. No subconjunto das instâncias $\{A, \dots, I\}$, o método obteve 31 novas melhores soluções e 14 empates, sem registrar qualquer resultado inferior. Já no conjunto SCP, o desempenho é ainda mais contundente: o LKH-CBM produziu 40 novas melhores soluções em 40 instâncias, superando integralmente os métodos concorrentes. Em termos percentuais,

isso significa que o método melhora 68,88% das instâncias do primeiro subconjunto e 100,00% das instâncias do segundo subconjunto.

Quanto à otimalidade, o LKH-CBM atingiu soluções ótimas para 42 instâncias do subconjunto $A, \dots, I$, coincidindo com o número de ótimos conhecidos na literatura para esse conjunto [Soares et al., 2020]. Para as três instâncias restantes, não há verificação formal de otimalidade na literatura. Ainda assim, o LKH-CBM obteve soluções com valores idênticos aos reportados nesses casos. Em comparação, o ILS encontrou 11 soluções ótimas, enquanto o ENS atingiu 14.

Esse resultado é particularmente relevante porque o *benchmark* considerado neste estudo reflete a própria evolução da literatura do CBM. As primeiras famílias de instâncias, utilizadas em estudos anteriores, já constituíam casos desafiadores; no entanto, a expansão posterior do *benchmark* introduziu instâncias substancialmente maiores, que elevaram o nível de dificuldade computacional do problema. Nesse cenário, o LKH-CBM não apenas preserva desempenho competitivo nas instâncias clássicas, mas desloca de forma clara a fronteira de resultados justamente nas instâncias de maior porte, isto é, nas situações em que a escalabilidade se torna mais importante do ponto de vista prático.

Esse comportamento reforça uma interpretação central deste estudo. O LKH-CBM produz um salto consistente de qualidade que não havia sido atingido pelos algoritmos mais recentes da literatura, mesmo aqueles concebidos especificamente para explorar intensificação e diversificação via meta-heurísticas.

Para verificar se a superioridade observada do LKH-CBM não decorre de flutuações aleatórias, foi aplicado o teste não paramétrico de Wilcoxon para amostras pareadas, comparando o método com ENS e ILS em termos de valor da função objetivo. Os resultados evidenciaram diferenças estatisticamente significativas entre o LKH-CBM e os demais algoritmos, tanto em relação ao ILS ($p = 3,84 \times 10^{-14}$) quanto ao ENS ($p = 1,21 \times 10^{-13}$), considerando um teste unilateral com hipótese alternativa de superioridade do LKH-CBM. Assim, rejeita-se a hipótese nula de equivalência entre os métodos, indicando que o LKH-CBM apresenta desempenho estatisticamente superior ao ILS e ao ENS ao nível de significância de 5%. Os resultados confirmam que o LKH-CBM é estatisticamente superior a ambos os algoritmos de referência, corroborando que os ganhos apresentados não se restringem a casos isolados do *benchmark*.

Do ponto de vista de tempo de execução, o LKH-CBM foi mais rápido que o ENS em 74 instâncias (87%), sem ocorrência de empates, e apresentou tempo superior nas 11 instâncias restantes (13%). Em relação ao ILS, o LKH-CBM obteve menor tempo de execução em 10 instâncias, sendo mais lento nas outras 75. Embora o método não domine sistematicamente esse critério, seus tempos permanecem plenamente compatíveis com o porte das instâncias avaliadas e com o nível de dificuldade do *benchmark*. Esse aspecto deve ser interpretado à luz do ganho substancial em qualidade de solução: o LKH-CBM não busca apenas manter competitividade temporal, mas sobretudo elevar de forma consistente o patamar dos melhores resultados conhecidos.

Do ponto de vista prático, esse resultado é bastante favorável ao LKH-CBM. ENS e ILS foram projetados com forte preocupação em eficiência computacional e, por isso, constituem referências particularmente exigentes também em tempo de execução. Ainda assim, o LKH-CBM permanece competitivo nesse critério, ao mesmo tempo em que supera amplamente os melhores resultados conhecidos do *benchmark*. Esse equilíbrio fortalece o valor aplicado do método, especialmente em cenários nos quais pequenas reduções adicionais no número de blocos podem produzir ganhos relevantes na aplicação final.

5.2. Impacto da reinicialização de ENS e ILS com soluções do LKH-CBM

Uma hipótese decorrente da análise dos resultados anteriores é a possibilidade de empregar uma camada meta-heurística adicional sobre as soluções produzidas pelo LKH-CBM, de

modo semelhante ao que ocorre nos métodos estado da arte baseados em k -opt. Para investigar essa hipótese, foram realizados experimentos complementares nos quais ENS e ILS passaram a ser inicializados a partir das soluções obtidas pelo próprio LKH-CBM.

O resultado foi particularmente significativo: em nenhuma instância as soluções fornecidas pelo LKH-CBM foram melhoradas por esses métodos. Esse achado mostra que o LKH-CBM não atua apenas como gerador de soluções iniciais promissoras, mas já produz soluções em um nível tal que estratégias adicionais de busca deixam de agregar valor prático ao *benchmark*. Além disso, ele reforça a tese central do estudo: no contexto do CBM, o principal avanço não está necessariamente na criação de novas camadas meta-heurísticas, mas na adoção de algoritmos mais adequados para a instância reduzida ao TSP. Assim, a superioridade do LKH-CBM decorre diretamente da força do algoritmo empregado e de sua integração à modelagem do problema, e não de refinamentos posteriores por procedimentos auxiliares.

6. Conclusão

Este estudo investigou a resolução do *Consecutive Blocks Minimization* (CBM) por meio de uma abordagem baseada na redução ao problema do caixeiro viajante e na aplicação do algoritmo *Lin-Kernighan-Helsgaun*, resultando no método LKH-CBM. Os resultados obtidos no *benchmark* consolidado da literatura mostraram que essa combinação é suficiente para deslocar o estado da arte do problema. No conjunto completo de 85 instâncias, o LKH-CBM produziu 71 novas melhores soluções, sem registrar resultados inferiores aos melhores valores previamente reportados. Além disso, o método identificou soluções ótimas para todas as 42 instâncias com ótimo conhecido, enquanto nas instâncias restantes obteve valores idênticos aos melhores já reportados. Esse desempenho foi particularmente expressivo nas instâncias de maior porte, nas quais a escalabilidade se torna mais relevante do ponto de vista prático.

Os experimentos complementares também mostraram que as soluções produzidas pelo LKH-CBM não foram melhoradas quando utilizadas como ponto de partida para meta-heurísticas, indicando que o avanço obtido neste estudo não depende da incorporação de camadas de busca adicionais. Esse resultado reforça a principal conclusão do estudo: no contexto do CBM, a adoção de um algoritmo mais forte para a instância reduzida ao TSP pode ser mais determinante do que a proposição de estruturas heurísticas progressivamente mais complexas. Como continuidade natural, estudos futuros podem investigar o comportamento do LKH-CBM em instâncias ainda maiores, bem como sua adaptação a variantes do problema ou a estratégias híbridas que preservem a simplicidade metodológica observada aqui, sem comprometer a qualidade das soluções.

7. Agradecimentos

O presente trabalho foi realizado com apoio da Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) - Código de Financiamento 001 e do Conselho Nacional de Desenvolvimento Científico e Tecnológico (CNPq), processo nº 407746/2025-0.

Referências

- Alizadeh, F., Karp, R. M., Newberg, L. A., e Weisser, D. K. (1995). Physical mapping of chromosomes: A combinatorial problem in molecular biology. *Algorithmica*, 13(1):52–76.
- Booth, K. S. e Lueker, G. S. (1976). Testing for the consecutive ones property, interval graphs, and graph planarity using pq-tree algorithms. *Journal of Computer and System Sciences*, 13(3): 335–379. ISSN 0022-0000.

- Christofides, N. (1976). Worst-case analysis of a new heuristic for the travelling salesman problem. Working Paper 62-75-76, Graduate School of Industrial Administration, Carnegie-Mellon University, Pittsburgh, PA.
- Dom, M. (2009). *Recognition, Generation, and Application of Binary Matrices with the Consecutive Ones Property*. Cuvillier, Göttingen.
- Fulkerson, D. e Gross, O. (1965). Incidence matrices and interval graphs. *Pacific journal of mathematics*, 15(3):835–855.
- Haddadi, S., Chenche, S., Cheraitia, M., e Guessoum, F. (2015). Polynomial-time local-improvement algorithm for consecutive block minimization. *Information Processing Letters*, 115(6):612–617. ISSN 0020-0190.
- Haddadi, S. (2002). A note on the np-hardness of the consecutive block minimization problem. *International Transactions in Operational Research*, 9(6).
- Haddadi, S. (2021). Iterated local search for consecutive block minimization. *Computers & Operations Research*, 131:105273. ISSN 0305-0548.
- Haddadi, S. (2022). Exponential neighborhood search for consecutive block minimization. *International Transactions in Operational Research*, 29(5):2855–2870.
- Haddadi, S. e Layouni, Z. (2008). Consecutive block minimization is 1.5-approximable. *Information Processing Letters*, 108(3):132–135.
- Hamming, R. W. (1950). Error detecting and error correcting codes. *The Bell system technical journal*, 29(2):147–160.
- Helsgaun, K. (2000). An effective implementation of the lin-kernighan traveling salesman heuristic. *European journal of operational research*, 126(1):106–130.
- Johnson, D., Krishnan, S., Chhugani, J., Kumar, S., e Venkatasubramanian, S. (2004). Compressing large boolean matrices using reordering techniques. In *Proceedings of the Thirtieth International Conference on Very Large Data Bases*, volume 30, p. 13–23, Toronto, Canada. VLDB Endowment.
- Kendall, D. (1969). Incidence matrices, interval graphs and seriation in archeology. *Pacific Journal of mathematics*, 28(3):565–570.
- Kou, L. T. (1977). Polynomial complete consecutive information retrieval problems. *SIAM Journal on Computing*, 6(1):67–75.
- Lemire, D. e Kaser, O. (2011). Reordering columns for smaller indexes. *Information Sciences*, 181(12):2550–2570.
- Lin, S. e Kernighan, B. W. (1973). An effective heuristic algorithm for the traveling-salesman problem. *Operations research*, 21(2):498–516.
- Soares, L. C., Reinsma, J. A., Nascimento, L. H., e Carvalho, M. A. (2020). Heuristic methods to consecutive block minimization. *Computers & Operations Research*, 120:104948. ISSN 0305-0548.